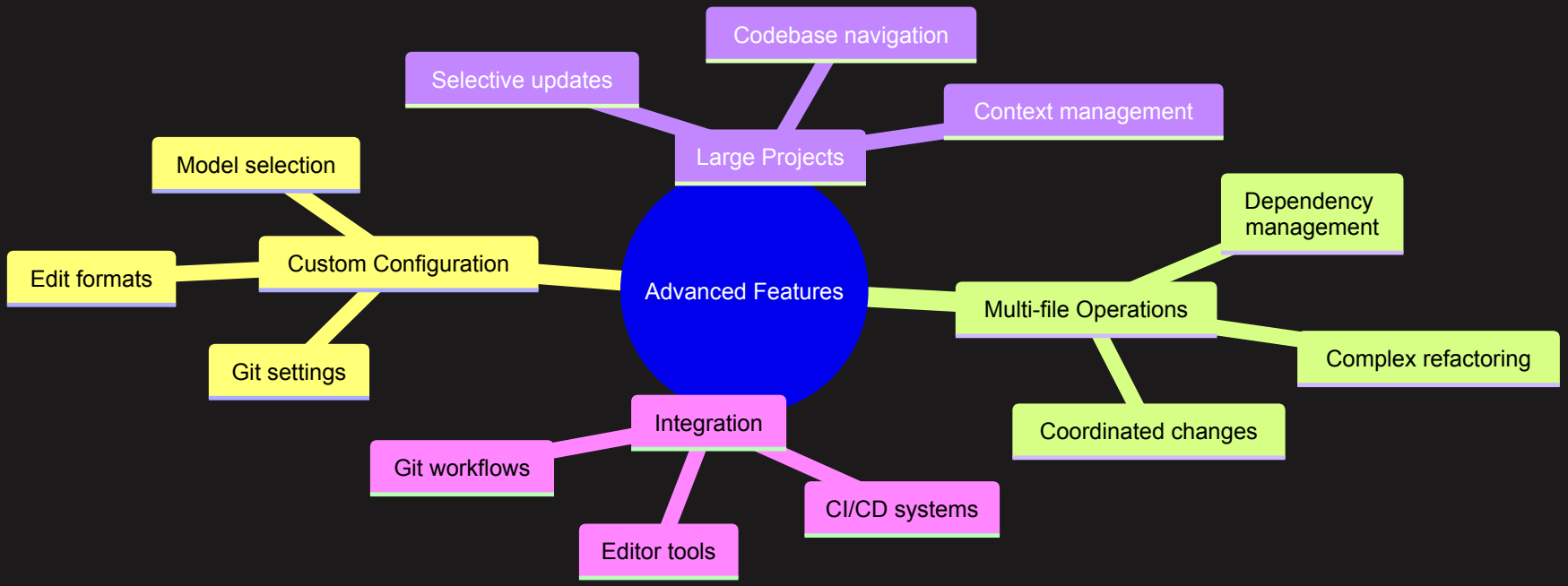
AI-Assisted Software Development

Module 4: Advanced aider Features

Mike Burton 4-6 November 2025

Advanced Features Overview



Model Selection and Configuration

You can select the model you want to use in a number of ways:

1. Command line:

```
aider --model gpt-4 .
```

2. Configuration file:

```
model: gpt-4
```

3. Environment variable:

```
export AIDER_MODEL=gpt-4
```

Working with Large Codebases

Strategies for managing large projects:

1. Selective File Addition:

```
aider src/core/*.py tests/core/*_test.py
```

2. Dynamic File Management:

```
> /add src/models/user.py
> /drop src/deprecated/*
```

3. Context Management:

```
> /files   # Check current files
> /drop unused_files.py
> /add relevant_file.py
```

Complex Refactoring

Example: Service Layer Extraction

```
> Let's refactor the user management system:
1. Extract database operations to a UserRepository class
2. Create a UserService class for business logic
3. Update controllers to use the new service
4. Move validation to a separate ValidationService
```

aider will:

1. Analyze dependencies
2. Create new files
3. Move and update code
4. Maintain consistency
5. Update imports

Advanced Git Integration

1. Custom Commit Messages:

```
/commit Refactor user management system to service architecture
```

2. Branch Management:

```
git checkout -b feature/service-refactor
aider .  # Make changes
git checkout main
```

3. Commit Hooks:

```
# In .git/hooks/pre-commit
#!/bin/sh
python run_tests.py
```

Custom Configurations

Example `~/.config/aider/config.yml`:

```yaml
# Model settings
model: gpt-4
temperature: 0.7

# Edit behavior
edit_format: diff
auto_commits: false

# Git settings
git_commit_prefix: "feat: "
git_commit_message_template: |
  {{prefix}}{{message}}

  Details:
  {{details}}

# Editor settings
editor: vim
```

Advanced Code Generation

1. Template-based Generation:

```
> Create a new microservice using our standard template:
- Controller layer
- Service layer
- Repository layer
- Unit tests
- API documentation
```

2. Pattern Replication:

```
> Apply the same error handling pattern used in
  UserService to OrderService and ProductService
```

Integration with Development Tools

1. Editor Integration:

```
# VS Code settings.json
{
  "terminal.integrated.profiles.linux": {
    "aider": {
      "path": "aider",
      "args": ["."]
    }
  }
}
```

2. CI/CD Pipeline:

```
# GitHub Actions example
jobs:
  aider-lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run aider checks
        run: aider --check .
```

Advanced Error Handling

1. Custom Error Strategies:

```
try:
    # Operation that might fail
except SpecificError as e:
    # Handle specific error
except Exception as e:
    # Generic error handling
finally:
    # Cleanup
```

2. Error Recovery:

```
> The last change caused test failures.
  Let's add proper error handling and fix the tests.
```

Performance Optimization

1. Context Management:

- Keep relevant files loaded
- Drop unnecessary files
- Focus on related code

2. Response Optimization:

- Be specific in requests
- Break down complex changes
- Review changes incrementally

Working with Tests

Advanced testing patterns:

1. Test Generation:

```
> Generate comprehensive tests for UserService:
- Unit tests
- Integration tests
- Edge cases
- Mocking examples
```

2. Test Updates:

```
> Update all affected tests after the service
  layer refactoring
```

Code Analysis and Refactoring

Advanced refactoring patterns:

1. Architecture Updates:

```
> Refactor the application to use the repository pattern:
1. Create abstract repository interfaces
2. Implement concrete repositories
3. Update services to use repositories
4. Add dependency injection
```

2. Performance Improvements:

```
> Optimize the database queries in UserRepository:
1. Add caching
2. Implement connection pooling
3. Add query optimization
```

Custom Commands and Workflows

Creating specialized workflows:

1. Documented Changes:

```
> Update the API endpoints and:
1. Add OpenAPI documentation
2. Update README
3. Generate API client
```

2. Migration Scripts:

```
> Create database migration for new user fields:
1. Add migration script
2. Add rollback script
3. Update models
```

Best Practices for Advanced Usage

1. Complex Changes:

- Break down into smaller tasks
- Review incrementally
- Test frequently

2. Code Organization:

- Maintain clear structure
- Use consistent patterns
- Document changes

3. Version Control:

- Use feature branches
- Write clear commit messages
- Review changes carefully

Recap: Advanced Features

- Custom configurations
- Complex refactoring
- Large project management
- Tool integration
- Advanced patterns
- Performance optimization
- Testing strategies

Next Steps

- Real-world applications
- Team workflows
- Project management
- Advanced patterns

Questions to Consider:

- How can you optimize your configuration?
- What complex refactoring could benefit your project?
- How can you integrate aider into your tools?